

NABİZ

Otonom Donanım-Farkındalıklı Dağıtık Sistemler Çekirdeği

Teknik Mimari ve Ekosistem Entegrasyonu Raporu

Yatırımcı ve Geliştirici Kavramsal Referans Dökümantasyonu

Geliştirici: Elif Nur Ayhan

Tarih: Haziran 2026

Durum: Üretime Hazır (Production-Ready)

1. Yönetici Özeti ve Vizyon

Günümüz yazılım mimarileri, üzerinde çalıştıkları donanımın fiziksel gerçekliğinden tamamen kopuktur. Bir işletim sistemi ısındığında veya güç kaybettiğinde, üzerindeki yazılımlar "hiçbir şey olmamış gibi" aynı agresif hızda bellek tüketmeye ve işlemciyi yormaya devam eder. Bu durum, sunucu çöküşlerine (BaaS), batarya ömrünün hızla tükenmesine (Edge) ve donanım ömrünün kısalmasına yol açar.

Nabız, yazılım ile donanım arasına zeka koyan otonom bir optimizasyon motorudur. Sadece reaktif (tepkisel) değil, proaktif (öngörücü) bir yaklaşımla, cihazın ısınacağını veya bataryasının tükeneceğini önceden hesaplayarak uygulamanın kendi mimarisini çalışma anında değiştirmesini sağlar.

2. Çekirdek Sistem Mimarisi

Sistem, birbirini tamamlayan üç temel sütun üzerinde yükselir:

2.1. Donanım İzleme ve Proaktif Yapay Zeka Motoru

Sistem, sadece o anki sıcaklık veya CPU yüküne bakmakla kalmaz. İçerisinde barındırdığı zaman serisi (time-series) analizi ve en küçük kareler yöntemi ile sıcaklık değişim ivmesini hesaplar.

Örnek Senaryo: İşlemci anlık olarak 55°C olabilir, ancak eğim çok yüksekse motor bunu algılar ve "15 saniye içinde 90°C'ye ulaşılacak, şimdiden önlem alıyorum" diyerek sistemi güvenli moda geçirir.

2.2. Dinamik Bellek Stratejileri (Memory Modes)

Uygulamanın bellek yönetimi (Allocation), anlık donanım sağlığına göre üç farklı modda çalışır:

Mod Adı	Donanım Durumu	Sistem Davranışı
Performance	Serin, Şarjda, Yük Hafif	Çöp toplayıcı (GC) baskılanır, RAM'den maksimum tahsisat yapılır. Tam hız.
Balanced	Standart Kullanım	Dengeli nesne yaratımı ve standart temizlik kuralları işletilir.
Eco/Thermal	Aşırı Isınma / Pil Kaybı	İşletim sisteminden RAM istenmez. "Object Pool" (Nesne Havuzu) kullanılarak sadece var olan bellek geri dönüştürülür.

3. İleri Seviye Özellikler ve Entegrasyonlar

3.1. JIT Motoru ve Yorumlayıcı Entegrasyon Köprüsü

Sistem kritik seviyede ısındığında bellek tasarrufu yapmakla kalmaz, kodun yürütülme katmanına (Runtime) da müdahale eder. JIT Köprüsü, ağır hesaplama gerektiren döngüleri tespit eder. Sistem **Eco/Thermal** moda geçtiğinde, bu döngüleri çalıştırmayı anında durdurur ve işlemciyi korumak için daha önceden hesaplanmış, önbelleğe alınmış (cached) veya güvenli (fallback) sonuçları döndürür.

3.2. Çevrimdışı-Öncelikli (Offline-First) Edge Optimizasyonu

Özellikle mobil cihazlar ve dış API çağrılarının maliyetli olduğu IoT cihazları için tasarlanmıştır. Cihaz ısındığında veya pili azaldığında, ağ çipini yormamak adına dış API'lere yapılacak veri aktarımlarını durdurur. Veriler yerel olarak (cihazda) sıraya alınır. Cihaz tekrar soğuduğunda ve şarja takıldığında, arka planda sessizce çalışan bir işçi biriken bu verileri topluca (batch) buluta senkronize eder.

3.3. Merkezi Telemetri ve BaaS Uyumu (Yük Dağıtımı)

Proje, FastAPI gibi modern backend mimarileriyle doğrudan entegre çalışacak şekilde tasarlanmıştır. Bir sunucu (node) kapasitesini aştığında ve aşırı ısınma sinyali verdiğinde, sistem çökmeyi beklemmez. Gelen HTTP isteklerini anında **503 Service Unavailable** koduyla reddederek korumaya alır. Bu sayede merkezi Yük Dağıtıcı (Load Balancer), trafiği anında filodaki diğer serin sunuculara yönlendirir.

4. Kıyamet Senaryosu (Doomsday) Direnci

Yapılan "Bellek Bombası" (Memory Bomb) ve Termal Kaçak (Thermal Runaway) stres testlerinde sistemin sınırları zorlanmıştır. İşlemcinin %100 yüke sokulduğu ve saniyeler içinde 100°C'ye ulaşılacağı simüle edildiğinde;

- Proaktif motor tehlikeyi saniyeler öncesinden hesaplayıp donanım bağlantılarını kilitlemiştir.
- Ağır iş yükleri JIT köprüsü tarafından saliseler içinde bypass edilmiştir.
- Uygulama içerisinden gelen binlerce nesnelik "Bellek Bombası" talebi, donanımı korumak adına sistem çökmeksizin (graceful degradation) absorbe edilmiş ve reddedilmiştir.

Sonuç: Hiçbir veri kaybı yaşanmamış ve işletim sistemi seviyesinde kilitlenme (Crash) engellenmiştir.

5. Ekosistem Entegrasyonu: Kardelen ve Yankı

Nabız, sadece bağımsız bir kütüphane değil, aynı zamanda daha geniş bir yazılım ekosisteminin otonom hayatta kalma (survival) çekirdeğidir. Mimarisi, özellikle **Yankı (Nexus BaaS)** ve **Kardelen** projelerinin temel altyapı ihtiyaçları gözetilerek tasarlanmıştır.

5.1. Yankı (Nexus BaaS) Filo Yönetimi

Yankı, dağıtık bir Backend-as-a-Service altyapısıdır. Nabız, Yankı'nın merkezi telemetri ve donanım-farkındalıklı yük dağıtım (Load Balancing) omurgasını oluşturur. Yankı sunucu filolarındaki (node) herhangi bir makine aşırı ısınma veya bellek darboğazı sinyali verdiğinde, Nabız proaktif olarak gelen HTTP isteklerini anında **503 Service Unavailable** ile reddeder. Bu otonom savunma sayesinde Yankı, çökmeleri beklemeden trafiği anında filodaki diğer serin ve uygun sunuculara aktararak %100 kesintisiz (High Availability) hizmet sunar.

5.2. Kardelen Yorumlayıcısı (Interpreter) ve JIT Kalkanı

Kardelen, yenilikçi ve yüksek performanslı bir programlama dilidir. Nabız'ın JIT entegrasyon köprüsü, doğrudan Kardelen'in yorumlayıcısı (interpreter) içine gömülü bir **Self-Optimizing LLM-Driven** kalkan olarak görev yapar. Donanım kritik sıcaklıklara ulaştığında, Kardelen üzerindeki ağır döngüler Nabız tarafından saliseler içinde bypass edilir. Bu sayede işlemciyi yormak yerine, daha önceden hesaplanmış güvenli sonuçlar (cache) veya LLM destekli optimize edilmiş çıktılar döndürülür. Nabız, Kardelen'i donanımın acısını hisseden ve buna göre kendi kod yürütme biçimini anında değiştirebilen yaşayan bir dile dönüştürür.